

S32K3XX COMMUNICATION MODULES: FLEXCAN WITH RTD

Automotive Systems & Applications Engineering Team
GPIS-BL, Automotive Processing



SECURE CONNECTIONS
FOR A SMARTER WORLD

PUBLIC

NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.





AGENDA

– S32K3xx CAN Module Introduction

- High level block overview and key features
- FlexCAN module clock resources between K1 and K3
- Operation mode overview
- Message buffers memory partition
- Legacy FIFO and enhanced FIFO
- Bit timing and other features

– RTD CAN Driver

- S32DS CT configuration steps
- Generated code and Key APIs

– Use Case Introduction

- Interrupt and poll method
- Legacy and enhanced FIFO filter table element configuration
- Bus off recovery process
- Queue feature

S32K3xx CAN Module

INTRODUCTION



SECURE CONNECTIONS
FOR A SMARTER WORLD

PUBLIC

NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.



FLEXCAN OVERVIEW – BLOCK DIAGRAM AND FEATURES

Bus Interface Unit (BIU)

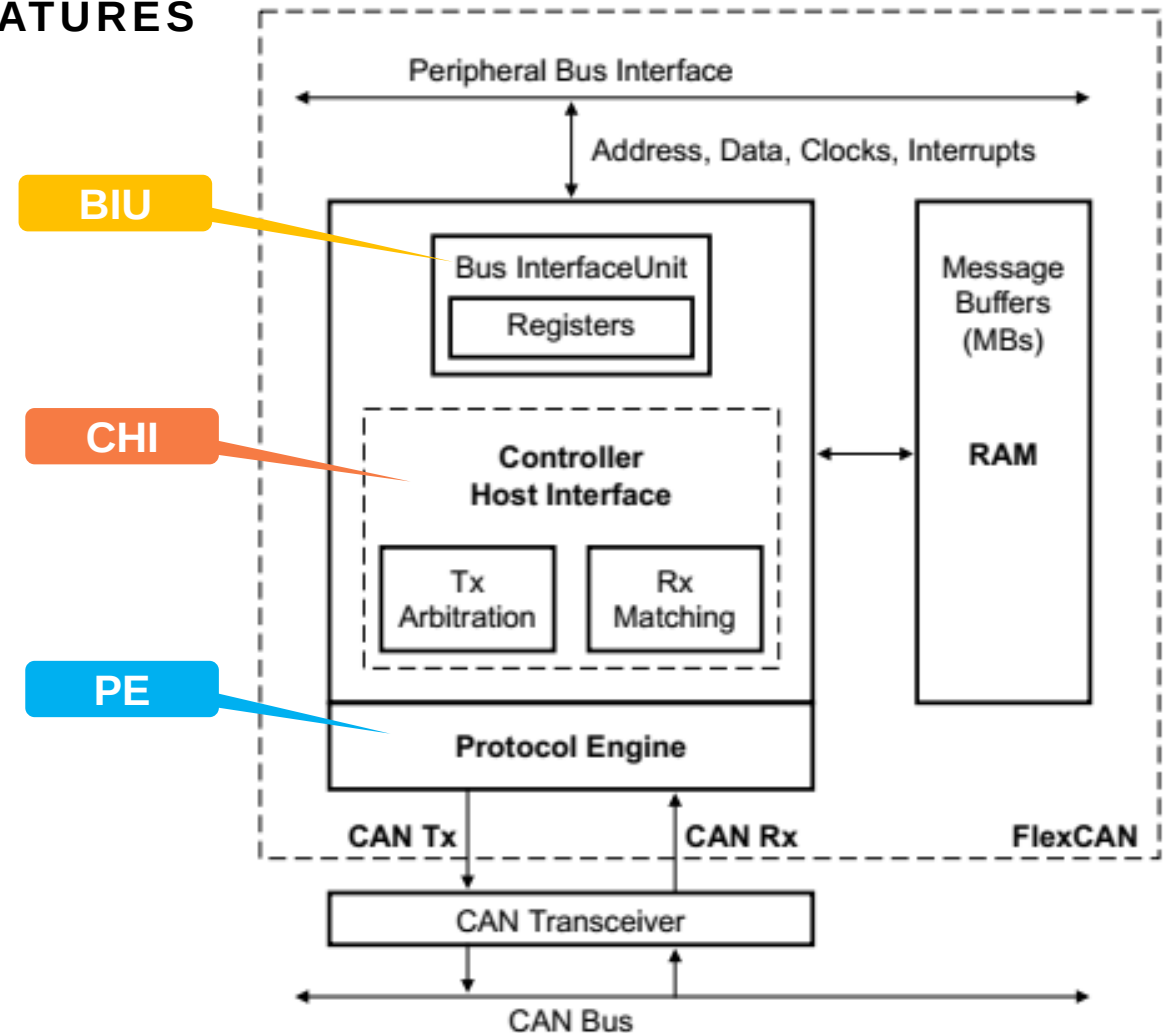
Allow clocks, address and data buses, interrupt outputs, DMA and test signals access to module.

Controller Host Interface (CHI)

Message buffer arbitration, matching algorithms

Protocol Engine (PE)

Request RAM access, error handling, detect CAN FD frame, validate received frame

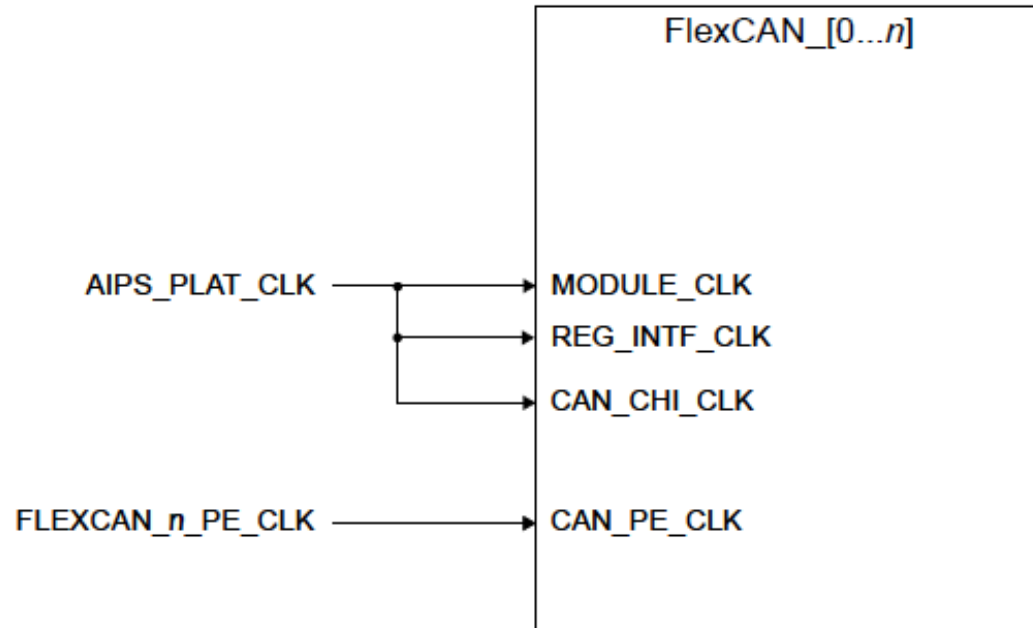


Some key features:

- Flexible mailboxes configurable to store 0 to 8-, 16-, 32-, or 64-bytes data length.
- RAM not used by reception or transmission structures can be used as general-purpose RAM space.
- Enhanced FIFO for CAN FD with storage capacity for up to 20 CAN FD frames and automatic internal pointer handling with DMA support.
- Programmable transmission priority scheme: lowest ID, lowest buffer number, highest priority.
- Maximum 96 mbs in CAN0 and 64 mbs in CAN1, 2. (S32K3x4).

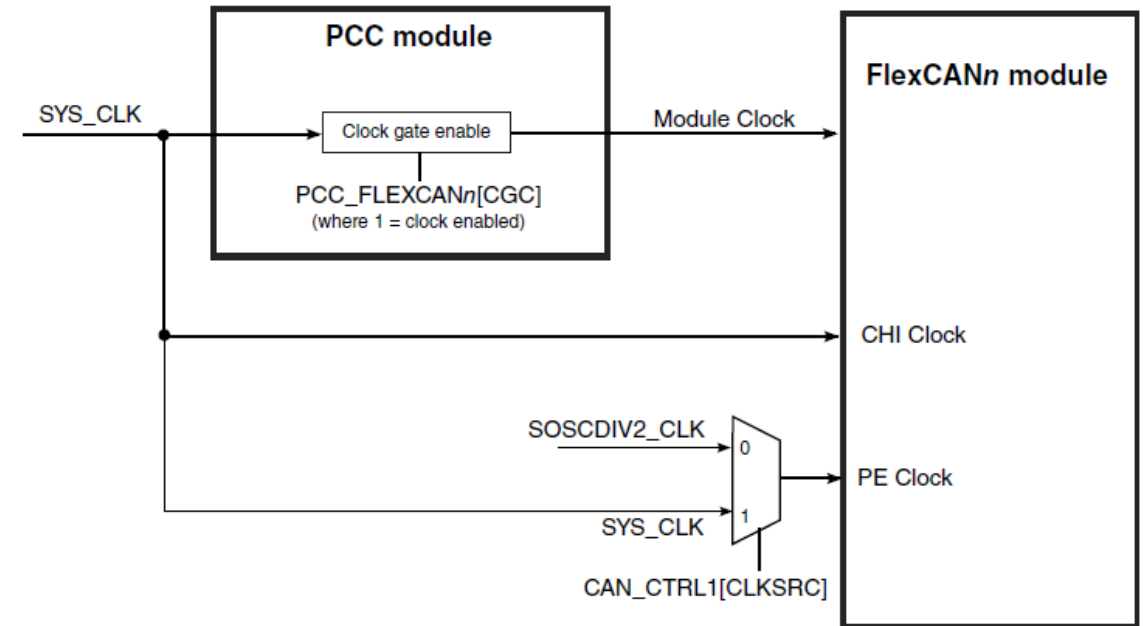
FLEXCAN OVERVIEW – CLOCK

S32K3



- AIPS_PLAT_CLK is clock resource of module clock
- CAN_PE_CLK can select the following three clock:
 1. AIPS_PLAT_CLK (typical 80Mhz)
 2. FIRC_CLK (48Mhz)
 3. FXOSC_CLK (8MHz ~ 40MHz)

S32K1



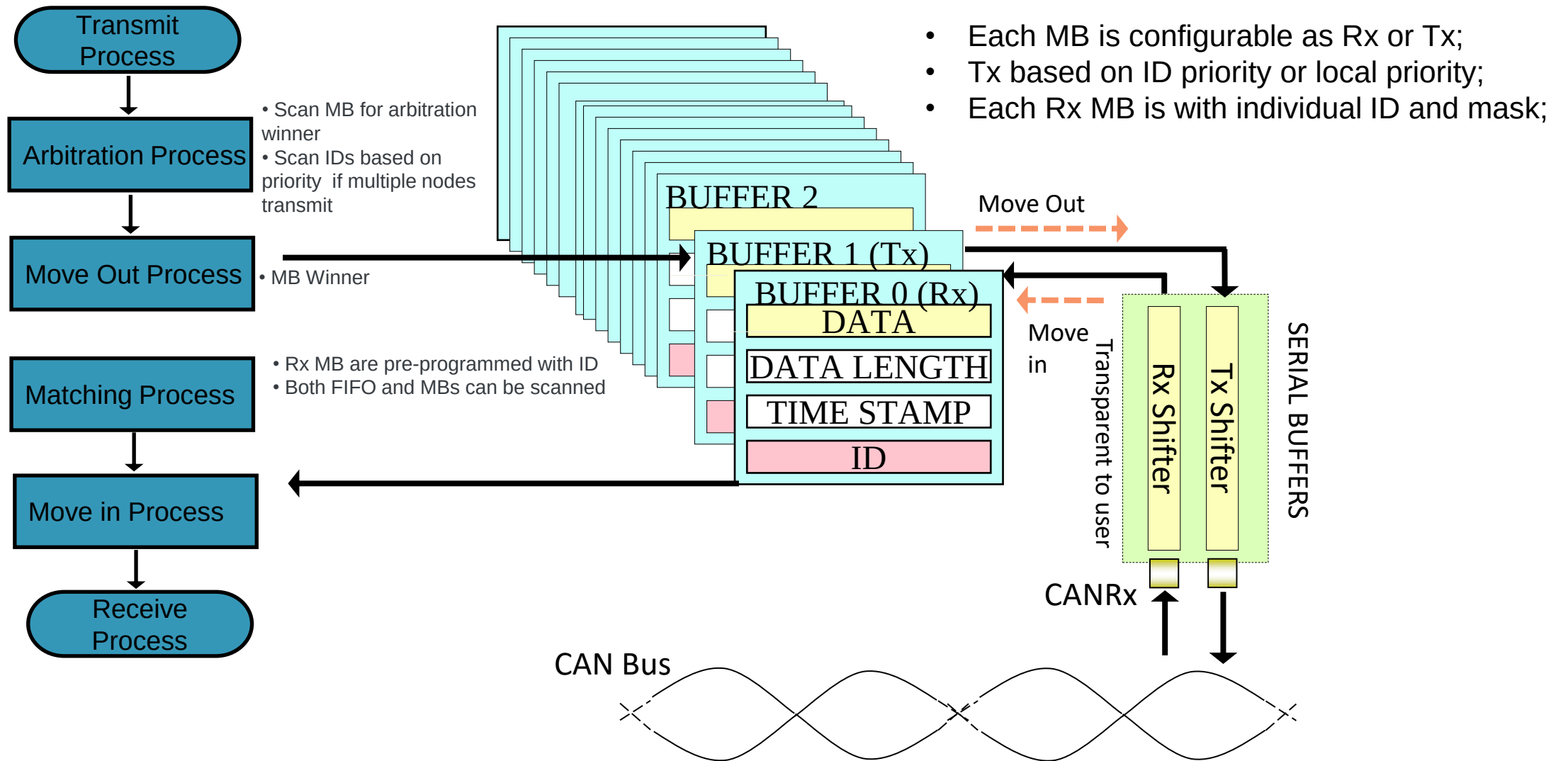
- SYS_CLK provide the module clock, CHI clock.
- CAN_PE_CLK can select between SOSCDIV2_CLK and SYS_CLK.
- Operational in VLPR mode, with 4MHz system clock, the fastest supported FlexCAN transfer rate is 250kbps

FLEXCAN OVERVIEW – OPERATION MODE

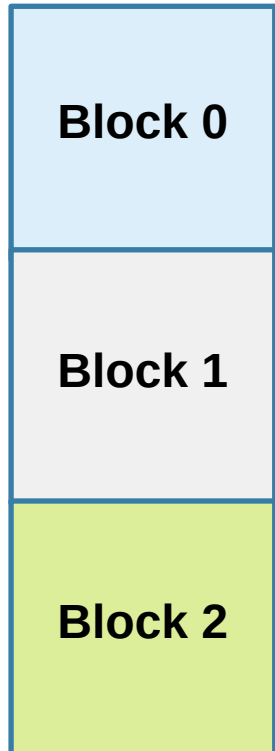
S32K3	S32K1
Normal mode(user or supervisor)	
Freeze mode	
Loop back mode	
Listen only mode	
CAN FD active mode	
Low-power operation	
Module disable mode	
N/A	Stop mode
N/A	Pretended Network mode

- Freeze mode is entered by MCR[HALT] and [FRZ], and it is necessary module not be in low-power mode. User should wait for MCR[FRZ_ACK] to be asserted to guarantee module is truly enter the freeze mode.
- Module disable mode is entered when MCR[MDIS] asserted and MCR[LPM_ACK]. Exit from this mode is done by negating MCR[MDIS].
- In module disable mode, the module issues a request to disable the clocks to the CAN Protocol Engine and Controller Host Interface submodules.

FLEXCAN BASICS - TRANSMIT, ARBITRATE, MOVE, MATCH AND RECEIVE



FLEXCAN FD MEMORY PARTITIONING



- FlexCAN FD memory is partitioned in blocks
 - Essentially 96 x 8-byte payload buffers can be supported
 - SoC can instantiate 0 to 2 blocks
- Each block is composed by the memory space equivalent to 32 x 16-byte MBs or 512-bytes (0x200)
- Each block can be configured to have message buffers (MBs) of a defined size in order to store messages with payload of 8-byte, 16-byte, 32-byte or 64-bytes. Message buffer data size region bit field:
 - **FDCTRL[MBDSRn] = 0bxx**
- The number of message buffers in a block depends upon payload size. Each block can be configured (by software) as follows:

RAM block	Number of MBs with 8 bytes (default range)	Size control bit field in FDCTRL register	Number of MBs of different sizes, per block
0	0 to 31	MBDSR0	MBDSR0=00, 32 MBs with 8 bytes payload MBDSR0=01, 21 MBs with 16 bytes payload MBDSR0=10, 12 MBs with 32 bytes payload MBDSR0=11, 7 MBs with 64 bytes payload

FLEXCAN – CAN FD MEMORY PARTITION

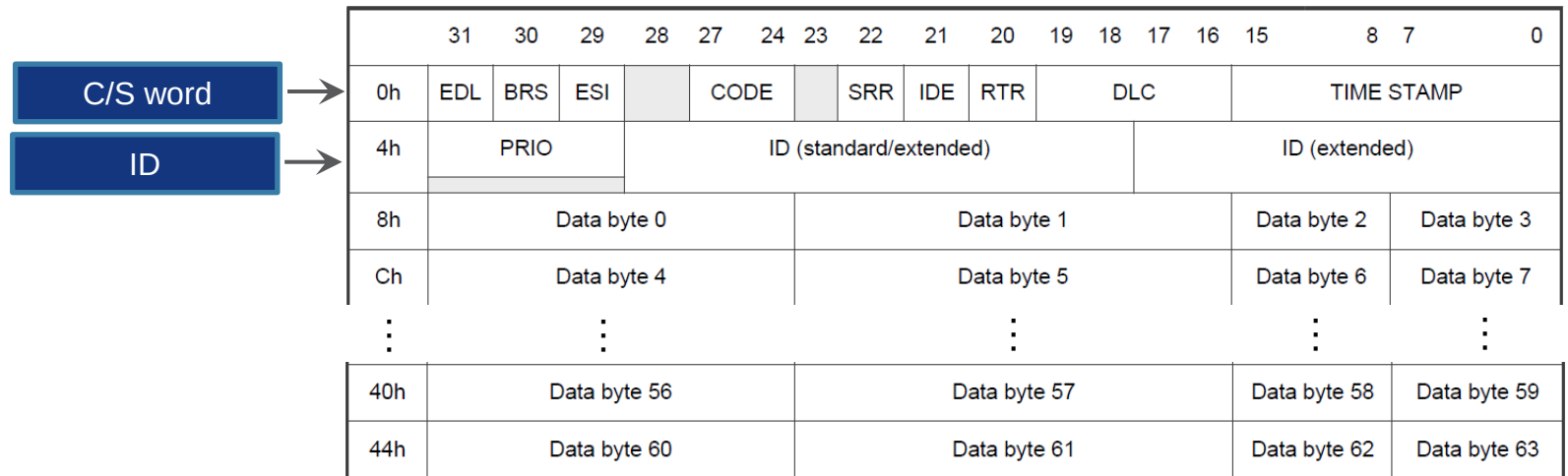
- When an 8-byte payload is selected:
 - Block R0: 0x0080~0x027F, 512bytes, allocate to MB0 ~ MB31.
 - Block R1: 0x0280~0x047F, 512bytes, allocate to MB32 ~ MB63.
 - Block R2: 0x0480~0x067F, 512bytes, allocate to MB64 ~ MB95.
- When a 64-byte payload is selected for all 3 blocks:
 - Block R0: 0x0080~0x0277, 504bytes, allocate to MB0 ~ MB6.
 - Block R1: 0x0280~0x0477, 504bytes, allocate to MB7 ~ MB13.
 - Block R2: 0x0480~0x0677, 504bytes, allocate to MB14 ~ MB20.
- Block Region 0, 1, 2 can be assigned different payload sizes.
- One memory block fits exactly 32 MBs with an 8-byte payload. For other possible payload sizes, empty memory may exist between the last MB in a block and the beginning of the next block. This empty memory corresponds to less than one MB and **must not be used**.

Payload Size	Max number of Msg Buffer per RAM block
8 bytes	32
16 bytes	21
32 bytes	12
64 bytes	7

FLEXCAN – CAN FD MEMORY BUFFER

- The only type of MB stores both extended (29-bit identifier) and standard (11-bit identifier) frames. Each individual MB is formed by 16, 24, 40, or 72 bytes, depending on payload: 8, 16, 32, or 64 data bytes, respectively.
- Address range from 0x80-67F occupy three blocks and up to 96 MBs(8 bytes payload).

RX CODE	Buffer State
0000b	Inactive
0100b	Empty
0010b	Full
0110b	Overrun
1010b	Ranswer
TX CODE	Buffer State
1000b	Inactive
1001b	Abort
1100b	Data
1100b	Remote
1110b	Tanswer



EDL — Extended Data Length, distinguishes CAN and CAN FD format frames.

BRS — Bit Rate Switch, enable the bit rate switch in CAN FD.

ESI — Error State Indicator, indicates if the transmitting node error state.

SRR — Substitute Remote Request, fixed recessive bit, used only in extended format.

IDE — ID Extended Bit, identifies standard or extended ID.

RTR — Remote Transmission Request, affects remote frames, part of the reception filter.

FLEXCAN – LEGACY RX FIFO

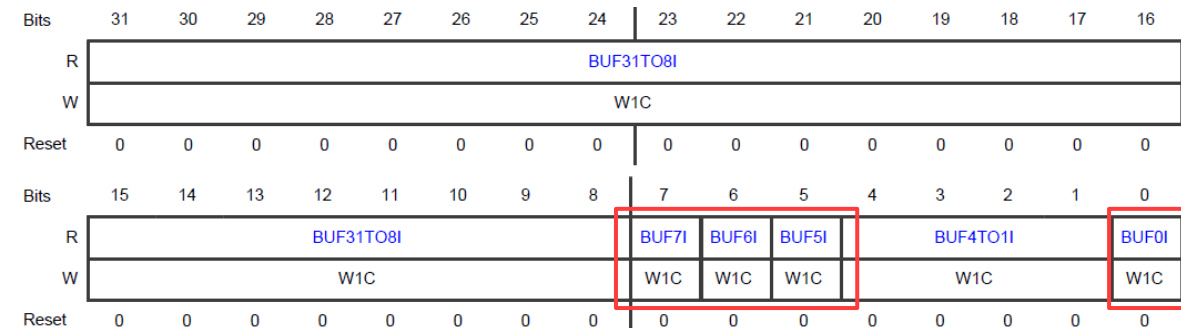
Features

- Enable by MCR[RFEN] bit, not compatible with CAN FD and enhanced FIFO.
- 6-message deep.
- CPU can read the received messages sequentially in the order they were received.
- If there are more than one frame in FIFO, when cpu read the output buffer, next buffer will be updated with RXFIR and reissue the interrupt.
- RXFIR will store the hit index.
- Support DMA operation.
- Total 4 filter format in legacy FIFO depending on MCR[IDAM].

```
/**< One full ID (standard and extended) per ID Filter Table element. */  
FLEXCAN_RX_FIFO_ID_FORMAT_A,  
/**< Two full standard IDs or two partial 14-bit (standard and extended)  
IDs per ID Filter Table element.*/  
FLEXCAN_RX_FIFO_ID_FORMAT_B,  
/**< Four partial 8-bit Standard IDs per ID Filter Table element.*/  
FLEXCAN_RX_FIFO_ID_FORMAT_C,  
/**< ALL frames rejected.*/  
FLEXCAN_RX_FIFO_ID_FORMAT_D
```

Interrupt flag

- IFInterrupt flag was Multiplexed in mb interrupt flag register IFLGA.
- IFLAG1[BUF5I]: Frame available in FIFO interrupt indication.
- IFLAG1[BUF6I]: Frame Rx FIFO Warning, frame is increased to five, meaning the FIFO is almost full.
- IFLAG1[BUF7I]: Overrun interrupt, frame lost.
- IFLAG1[BUF0I]: Empty the FIFO.
- Interrupt will be enabled in IMASK register.



When enable DMA operation

- IFLAG1[BUF7I] - [BUF6I] and [BUF1I] - [BUF4I] are not used.
- IFLAG1[BUF5I] will be used to generate a DMA request.
- IFLAG1[BUF1I] will be used to empty legacy FIFO.

FLEXCAN – LEGACY RX FIFO STRUCTURE

FIFO ENGINEE

- CPU can only read the output buffer to retrieve the data. Mb1-5 can not be read.
- Filter table elements table is adjacent to FIFO structure.
- Mbs which is used for filter table can not be used to transmission or reception.

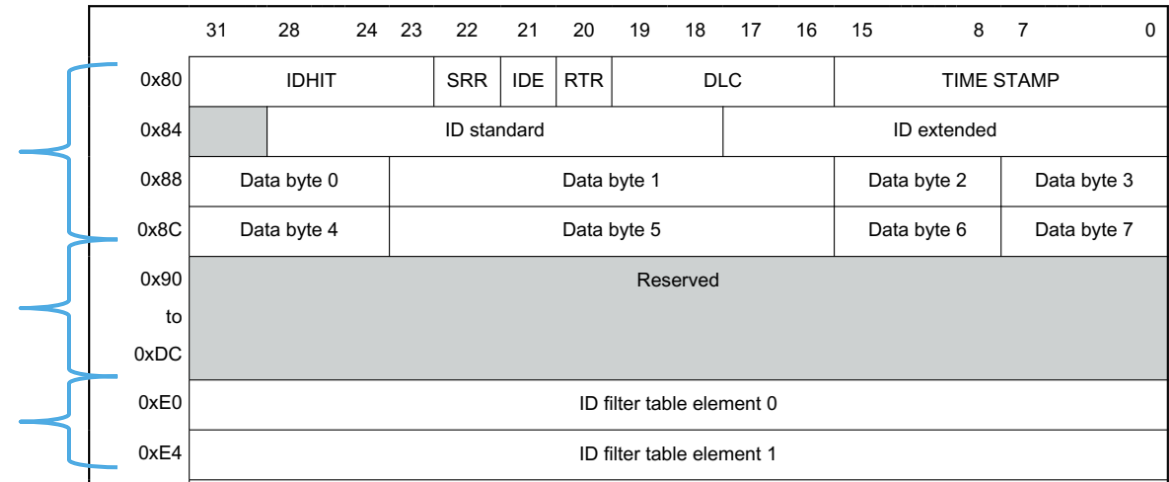
eg :

Format A, 8 filter elements, it will contain 8bytes RAM space(occupied mb6, 7). So, the normal transmission mb can only be set to at least mb8.

FIFO output

FIFO

FIFO filter table



- According to CTRL2[RFFN], up to 128 extended, 256 standard, or 512 partial (8 bit) IDs.
- Compared to K1 32 individual mask, K3 CAN0 support up to **96** individual masks (MCR[IRMQ] = 1) and a global mask(MCR[IRMQ] = 0).

Format A: only the 11 most significant bits (29 to 19) are used for matching standard frame. All bits are used In the extended frame.

Format B: The 11 most significant bits (29 to 19 and 13 to 3) are used for standard frame. All 14 bits are used to compare to the 14 most significant bits in extended frame.

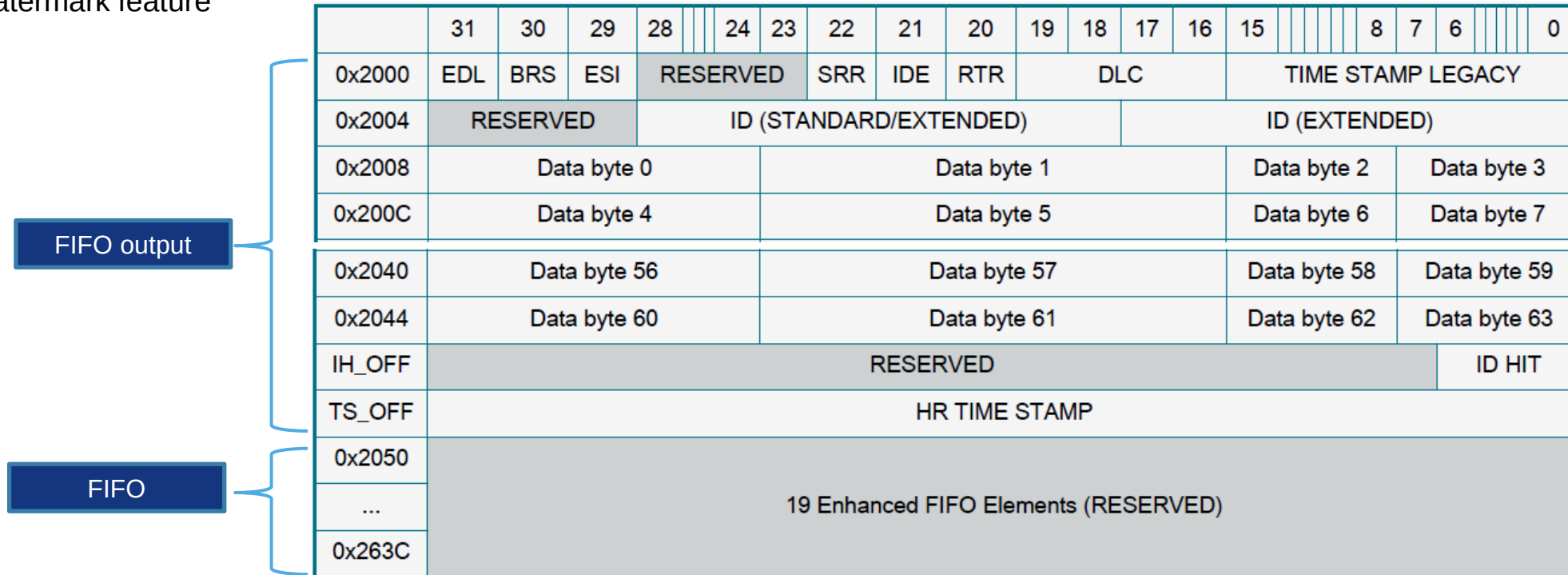
Format C: In both standard and extended frame formats, all 8 bits of the field are compared to the 8 most significant bits of the received ID.

Format	31	30	29	24	23	16	15	14	13	8	7	1	0
A	RTR	IDE	RXIDA (standard = 29–19, extended = 29–1)										
B	RTR	IDE	RXIDB_0 (standard = 29–19, extended = 29–16)					RTR	IDE	RXIDB_1 (standard = 13–3, extended = 13–0)			
C	RXIDC_0 (std/ext = 31–24)				RXIDC_1 (std/ext = 23–16)				RXIDC_2 (std/ext = 15–8)				RXIDC_3 (std/ext = 7–0)

FLEXCAN – ENHANCED RX FIFO STRUCTURE

FIFO ENGINE

- Support up to 20 CAN FD messages. Offset 0x2000h-0x204Fh, 80bytes memory size for output of the FIFO.
- Can not be active while enable legacy FIFO.
- DMA or interrupt request will be triggered while there are frames in FIFO or watermark reached.
- Watermark feature



Initialization process

1. Enter freeze mode and enable ERFCR[ERFEN];
2. Reset FIFO engine by write one to ERFSR[ERFCLR];
3. Clear all the interrupt flag and configure the filter table;
4. Set the DMA or interrupt enablement;
5. Leave freeze mode.

FLEXCAN – ENHANCED RX FIFO FILTER

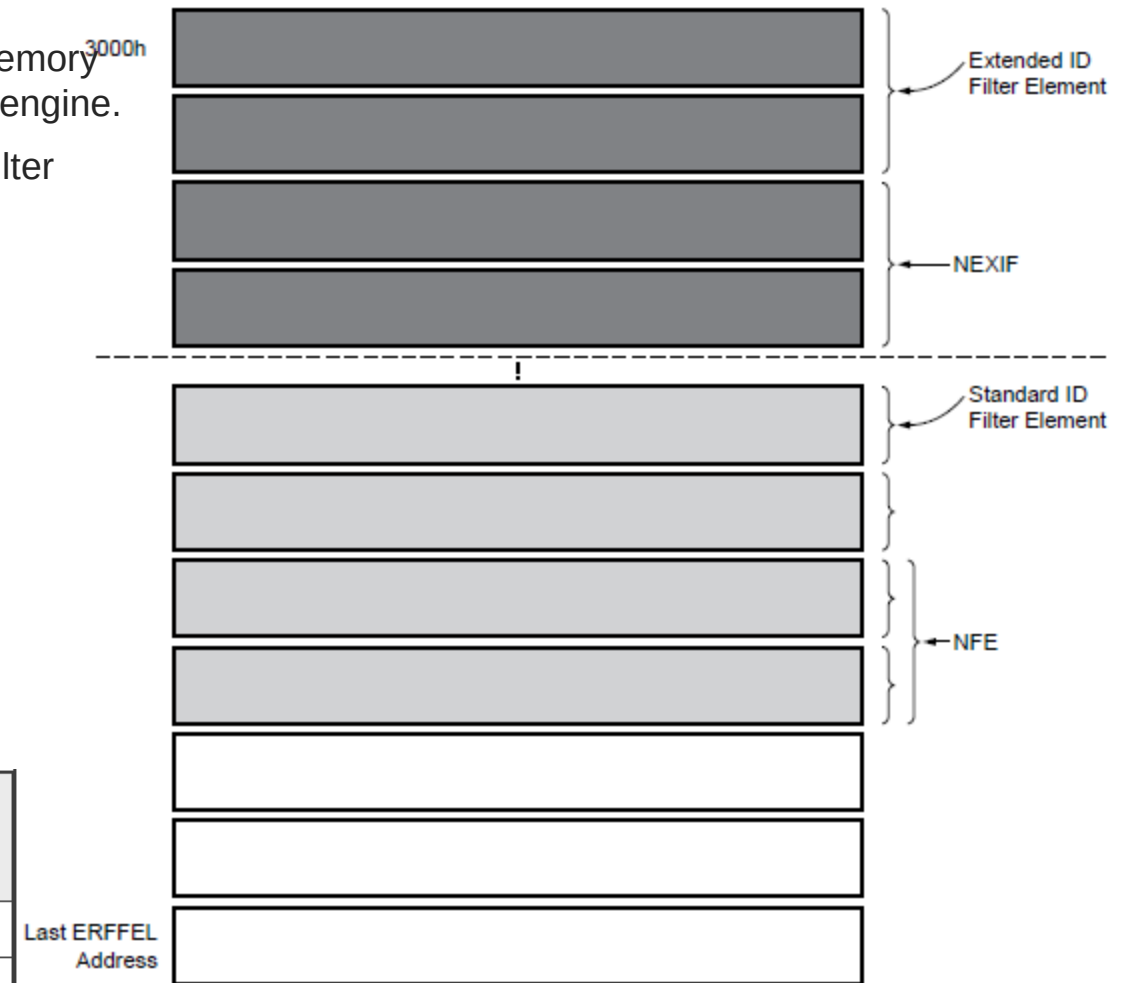
FILTER FEATURE

- When enhanced FIFO is active, FlexCAN scans the ERFFELn(0-127) memory region. If there is at least one element matched, store the frame in FIFO engine.
- If multiple filters match the incoming ID, IDHIT will be the first matching filter element.
- Each ERFFELn register can store one standard filter element.
- Each pair of ERFFELn register can store one extended filter element.
- ERFFELn offset address start from 0x3000.

The relationship between NEXIF and NFE.

- $NEXIF \leq NFE \text{ (up to 63)} + 1$
- Number of Standard ID Filter Elements = $2 \times (NFE - NEXIF + 1)$

NFE	Maximum number of Extended ID filter elements (NEXIF = NFE + 1)	Maximum number of standard ID filter elements (NEXIF = 0)
0	1	2
1	2	4
2	3	6



NFE: number of filter elements.
NEXIF: number of extended filter elements.

FLEXCAN – ENHANCED RX FIFO FILTER

STANDARD ID

CAN frame is base frame format(IDE=0).

1. ID + Mask:

31	30	29	28	27	26								16	15			12	11	10									0
FSCH=b00		Reserved		RTR filter	STD ID filter								Reserved				RTR mask	STD ID mask										

2. Range: ID1 – ID2:

$\text{STD ID Filter 1} \leq \text{ID} \leq \text{STD ID Filter 2}$

31	30	29	28	27	26									16	15			12	11	10									0
FSCH=b01		Reserved		RTRfilter	STD ID Filter2								Reserved				RTRmask	STD ID Filter1											

3. Two ID:

31	30	29	28	27	26								16	15			12	11	10									0
FSCH=b10		Reserved		RTR Filter 2	STD ID Filter2								Reserved				RTR Filter 1	STD ID Filter1										

EXTENDED ID

1. ID + Mask:

2. Range: ID1 – ID2:

3. Two ID:

PUBLIC

FLEXCAN – MAILBOX TX AND RX PRIORITY

- Tx Mailbox Priority

- CTRL1[LBUF]=1: lowest-number mailbox first.
- CTRL1[LBUF]=0: Highest-priority mailbox first:
 - MCR[LPRIOEN]=1: lowest “PRIO” (highest local priority) mailbox has highest priority.

Format	Mailbox arbitration value (35 bits)					
Standard (IDE = 0)	PRIO (3 bits)	Standard ID (11 bits)	RTR (1 bit)	IDE (1 bit)	- (18 bits)	- (1 bit)
Extended (IDE = 1)	PRIO (3 bits)	Extended ID[28:18] (11 bits)	SRR (1 bit)	IDE (1 bit)	Extended ID[17:0] (18 bits)	RTR (1 bit)

- MCR[LPRIOEN]=0: lowest CAN ID mailbox has highest priority.
- Same local priority and same ID: lowest-number mailbox first.

Format	Mailbox arbitration value (32 bits)				
Standard (IDE = 0)	Standard ID (11 bits)	RTR (1 bit)	IDE (1 bit)	- (18 bits)	- (1 bit)
Extended (IDE = 1)	Extended ID[28:18] (11 bits)	SRR (1 bit)	IDE (1 bit)	Extended ID[17:0] (18 bits)	RTR (1 bit)

- Rx Mailbox Priority

CTRL2[MRP]=0: matching starts from Rx FIFO and continues on Mailboxes.

CTRL2[MRP]=1: matching starts from Mailboxes and continues on Rx FIFO.

FLEXCAN – DATA COHERENCE

- Transmission abort mechanism

Request the abort of a pending frame.

CPU can be informed by the feedback mechanism.

To asserted this feature, must assert MCR[AEN].

CPU must write abort code(1001b) in CODE field.

- Mailbox inactivation.

Inactivate mailboxes neither participates in the arbitration process or matching process.

CPU must write inactive code(1000b or 0b) in CODE field.

A frame in the bus that matches the filtering of the inactivated Rx mailbox may be lost without notice, even if there are other mailboxes with the same filter

The inactivation automatically unlocks the mailbox

- Mailbox lock mechanism.

Triggered when reading the code and status word of a mb which code field is overrun or full.

Reading the free running timer will unlock the mbs.

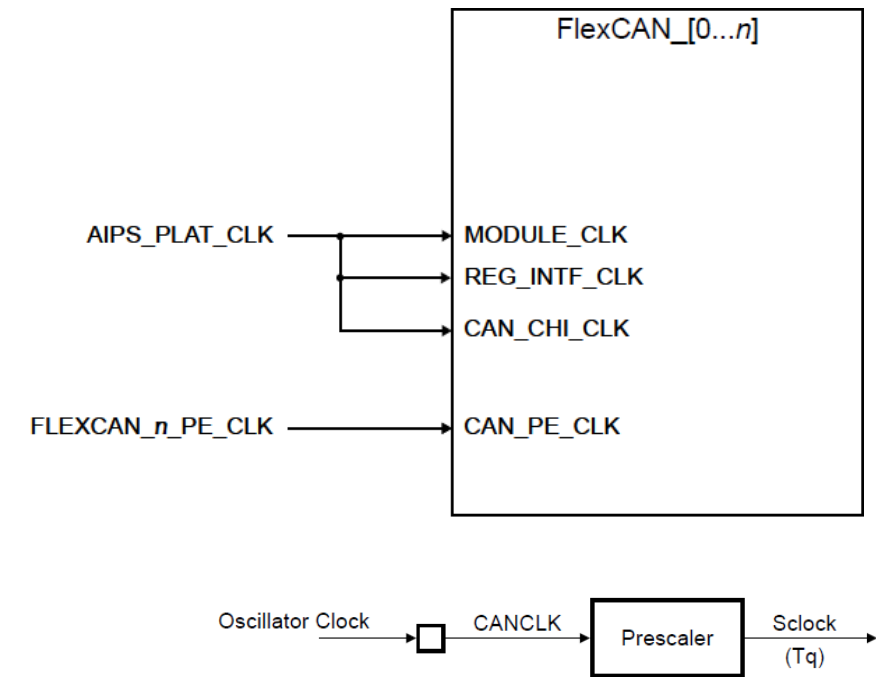
Only apply into RxMbs. FIFO output and Tx Mbs cannot be locked.

FLEXCAN - TIMESTAMP

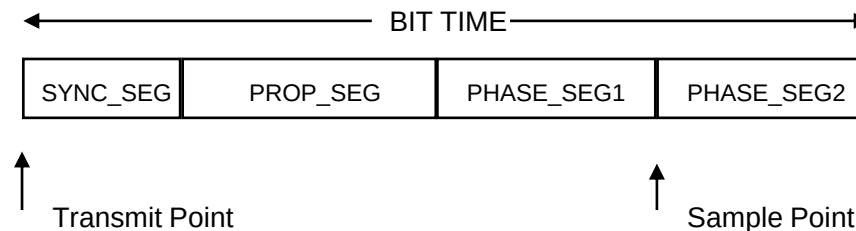
- Free running 16-bit Timer:
 - Can be read or written by CPU.
 - CTRL2[TIMER_SRC]: timer is clocked by CAN bit clock or by an external time tick (PIT).
 - CTRL1[TSYN]=1: timer can be reset whenever a message is received in MB0.
- High resolution 32-bit free running timer:
 - CTRL2[TSTAMPCAP] can turn on the high-resolution timer.
 - HR_TIME_STAMP0 stores the 32-bit HR time stamp associated with MB0, and so on.
- Message Buffer “TIME STAMP” can capture the following time base:
 - 16-bit CAN_TIMER.
 - Lower 16 bits of HR timer.
 - Upper 16 bits of HR timer.

FLEXCAN - PROTOCOL BIT TIMING

- FlexCAN Protocol Engine (PE_CLK) can be clocked from:
 - AIPS_PLAT_CLK (typical 80Mhz)
 - FIRC_CLK (48Mhz)
 - FXOSC_CLK (8MHz ~ 40MHz)
- CAN FD frame requires two bit rates – Arbitration and data phase
- Bit timing parameters are achieved by setting up the following registers:
 - When CTRL2[BTE]=0
 - Control register 1 (CTRL1): “Classical” CAN
 - CAN bit timing reg (CBT): Extended version of above register
 - CAN FD bit timing reg (FDCBT): Bit time associated with FD data phase
 - When CTRL2[BTE]=1
 - Enhanced CAN Bit Timing Prescalers (EPRS)
 - Enhanced Nominal CAN Bit Timing (ENCBT)
 - Enhanced Data Phase CAN Bit Timing (EDCBT)



- Bit time consists of following elements which are comprised from a variable number of time quanta



FLEXCAN - PROTOCOL BIT TIMING

	Sync_seg(Tq)	Prop_seg(Tq)	P_seg1(Tq)	P_seg2(Tq)	RJW
CTRL1		0-7*	0-7*	0-7*	0-3*
CBT		0-63*	0-31*	0-31*	0-31*
ENCBT	1 (Fixed)	0-255*		0-127*	0-127*
EDCBT		0-31*		0-15*	0-15*
FDCBT		0-31	0-7*	0-7*	0-7*

* Register value plus one will be actual value

- CTRL1 should be the configured for classical CAN frame bit timing.
- CBT should be the extended bit timing for classical CAN frame.
- FBCBT stores the CAN bit timing variables used in the data phase of CAN FD messages when the FDCTRL[FDRATE] is set. The contents of this register are not affected by soft reset.
- When set CTRL2[BTE], CAN FD nominal bit timing is configured in ENCBT, data bit timing in EDCBT. FDCBT will be read as 0.

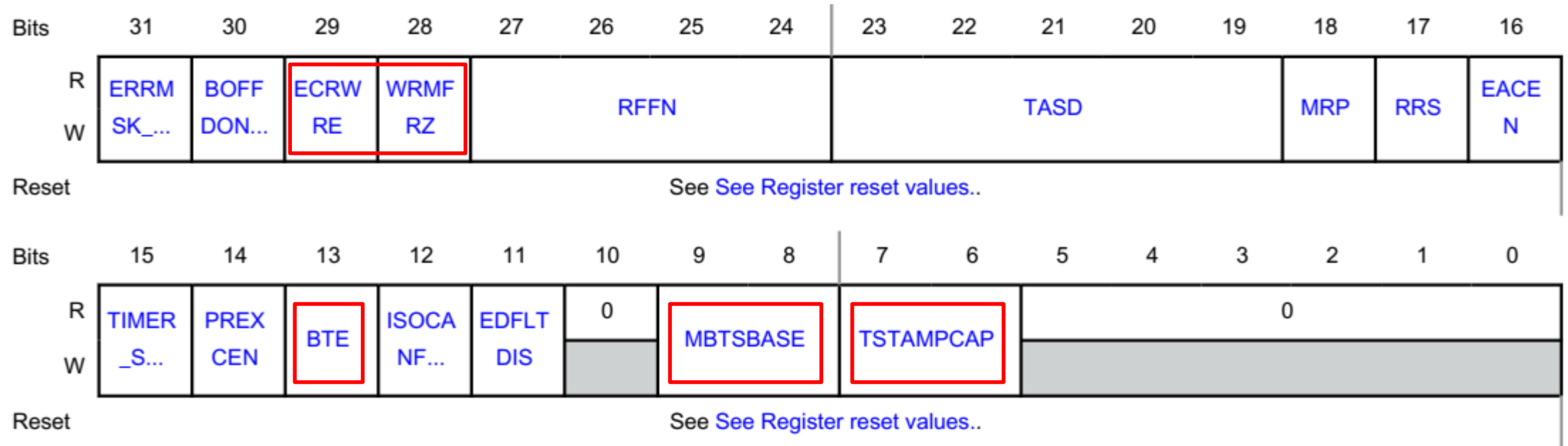
Tips

- In CAN FD mode, do not set CTRL1 register to configure bit timing.
- CBT register is an extended choice to configure bit rate and must be used in CAN FD mode while enable CBT[BTF]. CTRL1 register and CBT register are stored in the same register.

FLEXCAN - ECC

- FlexCAN memory ECC can correct 1-bit error (CE) and can detect 2-bit error (non-correctable error, NCE);
- Error Indication - Error Status register (ERRSR), an interrupt request can be issued to CPU.
 - NCE flag available.
 - Correctable Error flag and overrun flag;
- Error Reporting – error address and access requester, raw data, error bits can be read in following registers:
 - Error Report Address register (RERRAR)
 - Error Report Data register (RERRDR)
 - Error Report Syndrome register (RERRSYNR)
- Error Injection – error address, error data bits, error parity bits can be specified
- Error Response: (assuming MECR[NCEFAFRZ]=1)
 - FlexCAN stops operation and enters in Freeze mode;
 - CAN CRC error or bit error may occur on the bus;
 - CPU can read/process the error information, clear the error flags and change FlexCAN from Freeze mode to Normal mode

FLEXCAN – ECC CONFIGURATIONS



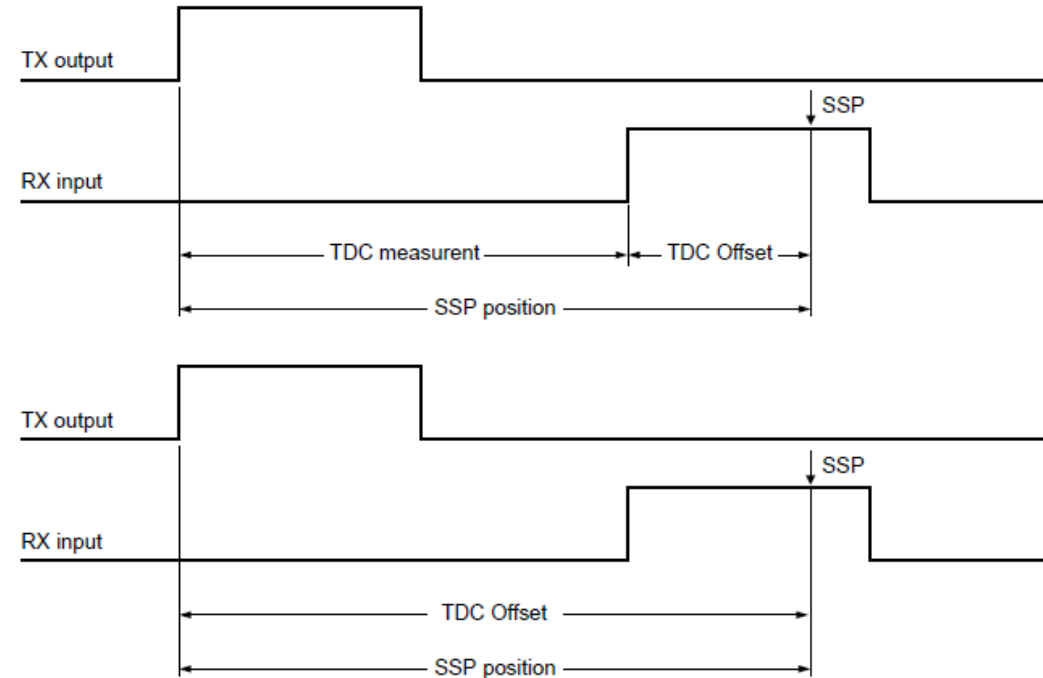
- ECRWRE: write protect to ECC configuration registers;
- WRMFRZ: enable access to hidden RAM in FlexCAN;
- BTE: Bit timing expansion enabled; (another set of registers for CAN bit timing)
- MBTSBASE: MB time stamp base;
- TSTAMPCAP: time stamp capture point;

FLEXCAN – TDC FEATRUE

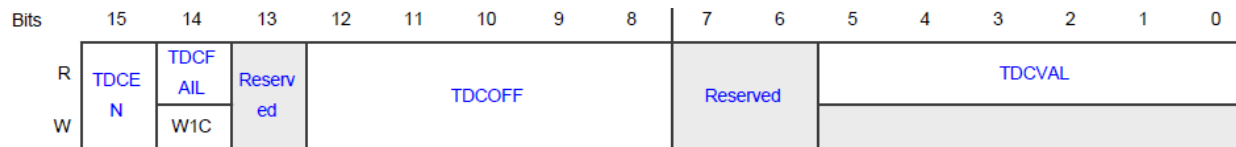
- Two TDC implementation method:

SSP position equals to measurement + offset or offset only(CTRL2[BTE] == 1, ETDC[ETDCEN] == 1, ETDC[TDMDIS] == 1).

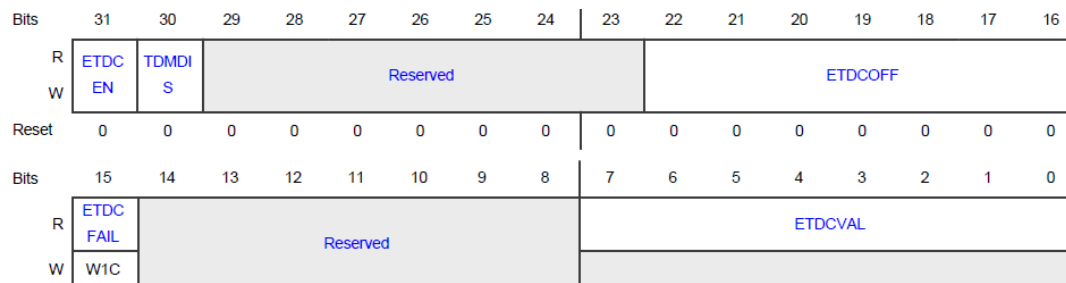
- Delay measurement is triggered by the EDL bit from recessive to dominant R0 bit.
- FDCTRL[TDCEN], [TDCOFF], [TDCVAL]: provide maximum 63 CANCLK.
- ETDC[ETDCOFF], ETDC[ETDCVAL]: provide maximum 255 CANCLK.



- FDCTRL



- ETDC



Difference and key features between K3 and K1

Features	S32K1xx	S32K3
CAN instance	Up to 3 FlexCAN (FD)	6 FlexCAN all support FD
Message buffer (16 bytes)	16, 32, 64 (S32K14xW) Message buffers	32, 64, 96 (CAN0) Message buffers
Time stamp	16-bit free running timer	32-bit high resolution time stamp
FIFO	6 mb legacy FIFO only support CAN 2.0 frame	20 mbs enhanced FIFO support CAN FD frame
PNET	Support Pretended Network	No longer supported Peripheral Chips Partial Networking
DMA	Support in legacy FIFO	Support in legacy/enhanced FIFO
ECC	Non-ECC memory	RAM ECC memory for mb

REAL-TIME DRIVER (RTD CAN DRIVER)



SECURE CONNECTIONS
FOR A SMARTER WORLD

PUBLIC

NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.



CONFIGURE TOOL

1. Add pins



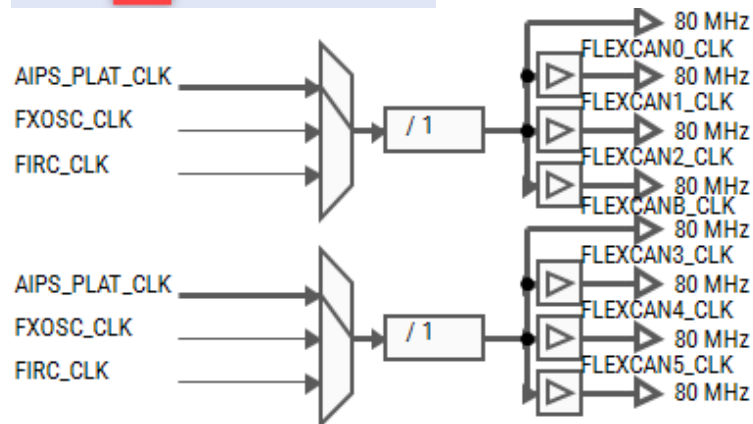
Tree view showing peripheral pin configuration:

- > ☐ FLEXIO
- > ☐ FlexCAN_0
- > ☐ FlexCAN_1
- > ☐ FlexCAN_2
- > ☐ FlexCAN_3
- ✓ ☒ FlexCAN_4
 - ✓ ☒ rxd <- [L3] PTE14 / 5 pins
 - ✓ ☒ txd -> [N1] PTE3 / 5 pins
- ✓ ☒ FlexCAN_5
 - ✓ ☒ rxd <- [P15] PTC26 / 5 pins
 - ✓ ☒ txd -> [R16] PTC27 / 5 pins
- > ☐ JTAGC
- > ☐ LCU_0
- > ☐ LCU_1

Routing Details for BOARD_InitPins (9):

#	Peripheral	Signal	Arrow	Routed pin/signal	Label	Identifier	Direction	Slew Rate
L3	FlexCAN_4	rxd	<-	[L3] PTE14		n/a	Input	n/a
N1	FlexCAN_4	txd	->	[N1] PTE3		n/a	Output	n/a
P15	FlexCAN_5	rxd	<-	[P15] PTC26		n/a	Input	n/a
R16	FlexCAN_5	txd	->	[R16] PTC27		n/a	Output	n/a

2. Configure the clocks



3. Add peripheral



Drivers

Select component

☐ Show only components in toolchain project ☒ Show only latest component versions

Configuration component	Component description	Category	Require
▲ Crc_Ip	Cyclic Redundancy Check Driver	Drivers	platfc
▲ Dma_Ip	Direct Memory Access	Drivers	platfc
▲ Emios_Gpt	EMIOS_GPT IPL Configuration	Drivers	platfc
▲ Emios_Icu	EMIOS Driver	Drivers	platfc
▲ Emios_Mcl_Ip	EMIOS	Drivers	platfc
▲ Emios_Ocu	EMIOS OCU Configuration	Drivers	platfc
▲ Emios_Pwm	Emios Pwm driver	Drivers	platfc
▲ FlexCAN	IP FlexCAN Driver	Drivers	platfc
▲ Flexio_I2c	FLEXIO_I2C	Drivers	platfc
▲ Flexio_Lin	Flexio Lin Configuration	Drivers	platfc
▲ Flexio_Mcl_Ip	FLEXIO	Drivers	platfc

CONFIGURE TOOL

- 1. Add/remove the used **can instance**: FlexCAN0 – 5.
- 2. Configure the **parameter** of module:
 [Name]: will be the name of configure structure.
 [Number Of MB] : 96(CAN0), 64(CAN1,2), 32(others), assuming 8 bytes payload.
 [Payload Size] : 8, 16, 32, 64bytes.
 [Rx FIFO Transfer Type] : interrupt, polling, DMA.

 [FlexCAN Protocol Clock] : Input the clock number in FlexCAN module.
- Configure **bit rate**
 CAN arbitration phase bit rate. Note the value which fill in here is the exact value in corresponding segment, register value will subtract 1.
- Configure **data bit rate**
 CAN data phase bit rate.

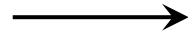
1

2

3

CONFIGURE TOOL

- Configure **data bit rate**
CAN data phase bit rate. Some limitation must be followed in protocol or reference manual. The minimum number of time quanta per CAN bit must be 8.



The screenshot shows the 'FlexCAN bitrate' and 'FlexCAN CBT bitrate' configuration panels. The 'FlexCAN bitrate' panel is expanded, showing fields for Name, Synchronization segment, Propagation segment, Phase segment 1, Phase segment 2, Prescaler division factor, Resync jump width, Bitrate (Kbps), and Sampling point (%). The 'FlexCAN CBT bitrate' panel is also expanded, showing similar fields. Below these, the 'Callback' and 'Error Callback' sections are visible, with fields for 'Can4_MB_RXTXCallback' and 'Can4_Error_Callback'. The 'Enable/Disable the Enhanced Rx FIFO feature' checkbox is unchecked.

FlexCAN bitrate	
Name	FlexCAN_Ip_TimeSeg4
Synchronization segment	1
Propagation segment	5
Phase segment 1	2
Phase segment 2	2
Prescaler division factor	16
Resync jump width	2
Bitrate (Kbps)	500
Sampling point (%)	80

FlexCAN CBT bitrate	
Name	FlexCAN_Ip_TimeSegCBT0
Synchronization segment	1
Propagation segment	7
Phase segment 1	6
Phase segment 2	2
Prescaler division factor	5
Resync jump width	2
Bitrate (Kbps)	1,000
Sampling point (%)	87.5

Callback	
Callback	Can4_MB_RXTXCallback

Error Callback	
Error Callback	Can4_Error_Callback
The number of enhanced ID filter elements	0
The number of enhanced extended ID filter elements	0
The number of enhanced Rx FIFO watermarks/The number of Message Buffers Transferred by DMA	0
Enable/Disable the Enhanced Rx FIFO feature	☐

- **Callback function** and enhanced **FIFO** setting
Enable or disable enhanced FIFO, configure the filter table elements number, extended filter elements number and watermark of FIFO engine.



Click 'Update Code' to generate code after configuration



The screenshot shows the bottom of the configuration tool interface. It includes a dropdown menu with the text 'S32K3_RTD_Training_CAN', a home icon, a warning icon, and a red-bordered button labeled 'Update Code' with a dropdown arrow.

GENERATED CODE

- RTD

- ▼ include

- > .h FlexCAN_Ip_DeviceReg.h
 - > .h FlexCAN_Ip_HwAccess.h
 - > .h FlexCAN_Ip_Irq.h
 - > .h FlexCAN_Ip_TrustedFunctions.h
 - > .h FlexCAN_Ip_Types.h
 - > .h FlexCAN_Ip_Wrapper.h
 - > .h FlexCAN_Ip.h

- ▼ SRC

- > .c FlexCAN_Ip_HwAccess.c
 - > .c FlexCAN_Ip_Irq.c
 - > .c FlexCAN_Ip.c

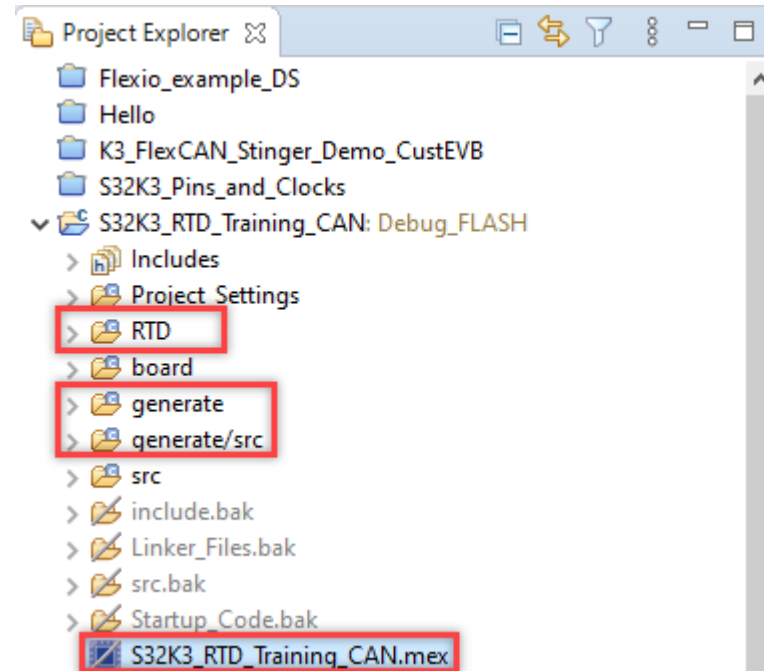
- ▼ generate

- ▼ include

- > .h Clock_Ip_Cfg_Defines.h
 - > .h FlexCAN_Ip_Cfg.h
 - > .h FlexCAN_Ip_CfgDefines.h
 - > .h FlexCAN_Ip_Sa_BOARD_InitPeripherals_PBcfg.h

- ▼ generate/src

- > .c FlexCAN_Ip_Sa_BOARD_InitPeripherals_PBcfg.c



```
/* FlexCAN State Structure used By driver
User should not modify this structure */
Flexcan_Ip_StateType FlexCAN_State0;
/* FlexCAN State Structure used By driver
User should not modify this structure */
Flexcan_Ip_StateType FlexCAN_State1;
```

```
const Flexcan_Ip_ConfigType FlexCAN_Config4 = {
    /* Number Of Message Buffer used .max_num_mb */
    (uint8)32,
    /*Can Hw filter count* .num id filters*- aici exista
    variatna sa generezi toate filtrele si sa referentiezi tu
    in cod */
    FLEXCAN_RX_FIFO_ID_FILTERS_8,
    /* Legacy FIFO ENABLED .is_rx_fifo_needed*/
    (boolean)FALSE,
    #if (FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO == STD_ON)
    /* The number of standard ID filter elements */
    0U,
    /* The number of extended ID filter elements */
    0U,
    /* The number of enhanced Rx FIFO watermark */
    0U,
    /* The Enhanced Rx FIFO feature is enabled or not. */
    (boolean)FALSE,
    #endif
    FLEXCAN_NORMAL_MODE,
```

GENERATED CODE

• FLEXCAN structure – in FlexCAN_Ip_Types.h

1

```
typedef struct
{
    uint32 max_num_mb;

    Flexcan_Ip_RxFifoIdFilterNumType num_id_filters;

    boolean is_rx_fifo_needed;

    #if (FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO == STD_ON)
    uint32 num_enhanced_std_id_filters;

    uint32 num_enhanced_ext_id_filters;

    uint32 num_enhanced_watermark;

    boolean is_enhanced_rx_fifo_needed;

    #endif /* FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO */
    #if (FLEXCAN_IP_FEATURE_HAS_TS_ENABLE == STD_ON)
    Flexcan_Ip_TimeStampConfigType time_stamp;
    #endif /* FLEXCAN_IP_FEATURE_HAS_TS_ENABLE */
    Flexcan_Ip_ModesType flexcanMode;

    uint32 ctrlOptions;
    Flexcan_Ip_PayloadSizeType payload;
    boolean fd_enable;
    #if (FLEXCAN_IP_FEATURE_HAS_PE_CLKSRC_SELECT == STD_ON)
    boolean is_pe_clock;
    #endif
    boolean enhCbtEnable;
    boolean bitRateSwitch;
    Flexcan_Ip_TimeSegmentType bitrate;
    Flexcan_Ip_TimeSegmentType bitrate_cbt;
    Flexcan_Ip_RxFifoTransferType transfer_type;
    #if (FLEXCAN_IP_FEATURE_HAS_DMA_ENABLE == STD_ON)
    uint8 rxFifoDMAChannel;
    #endif /* FLEXCAN_IP_FEATURE_HAS_DMA_ENABLE */
    FlexCAN_Ip_CallbackType Callback;
    FlexCAN_Ip_ErrorCallbackType ErrorCallback;
} Flexcan_Ip_ConfigType;
```

2

```
typedef struct FlexCANState
{
    Flexcan_Ip_MBhandleType mbs[FLEXCAN_IP_FEATURE_MAX_MB_NUM];

    #if (FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO == STD_ON)
    Flexcan_Ip_MBhandleType enhancedFifoOutput;
    #endif /* FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO */
    void (*callback)(uint8 instance,
                    Flexcan_Ip_EventType eventType,
                    uint32 buffIdx,
                    const struct FlexCANState *driverState
                    );
    void *callbackParam;

    void (*error_callback)(uint8 instance,
                          Flexcan_Ip_EventType eventType,
                          uint32 u32ErrStatus,
                          const struct FlexCANState *driverState
                          ); /*< Error IRQ handler callback
    void *errorCallbackParam;

    #if (FLEXCAN_IP_FEATURE_HAS_DMA_ENABLE == STD_ON)
    uint8 rxFifoDMAChannel;
    #if (FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO == STD_ON)
    uint32 u32NumOfMbTransferByDMA;
    #endif /* (FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO == STD_ON) */
    #endif /* (FLEXCAN_IP_FEATURE_HAS_DMA_ENABLE == STD_ON) */
    Flexcan_Ip_RxFifoTransferType transferType;
    boolean bIsLegacyFifoEn;
    #if (FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO == STD_ON)
    boolean bIsEnhancedFifoEn;
    #endif /* (FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO == STD_ON) */
    uint32 u32MaxMbNum;
    boolean isIntActive;
} Flexcan_Ip_StateType;
```

```
typedef struct
{
    Flexcan_Ip_MsgBuffType * pMBmessage;
    volatile Flexcan_Ip_MbStateType state;
    #if FLEXCAN_IP_FEATURE_HAS_DMA_ENABLE
    volatile boolean isDmaBusy;
    #endif /* FLEXCAN_IP_FEATURE_HAS_DMA_ENABLE
    boolean isPolling;
    boolean isRemote;
    uint32 time_stamp;
} Flexcan_Ip_MBhandleType;

typedef struct
{
    uint32 cs;
    uint32 msgId;
    uint8 data[64];
    uint8 datalen;
    uint8 id_hit;
    uint32 time_stamp;
} Flexcan_Ip_MsgBuffType;
```

Three mb state :

FLEXCAN_MB_IDLE,
FLEXCAN_MB_RX_BUSY
FLEXCAN_MB_TX_BUSY

KEY FUNCTION – INITIALIZATION/MODE SET

- Flexcan_Ip_StatusType **FlexCAN_Ip_Init_Privileged** (uint8 Flexcan_Ip_u8Instance, Flexcan_Ip_StateType * Flexcan_Ip_pState, const Flexcan_Ip_ConfigType * Flexcan_Ip_pData)
 - This function will initialize FlexCAN module and leave the module in freeze mode.
- Flexcan_Ip_StatusType **FlexCAN_Ip_Deinit_Privileged** (uint8 instance)
 - Disable all the interrupt, enter freeze mode and shut down the CAN module, clear the state structure.
- Flexcan_Ip_StatusType **FlexCAN_Ip_SetStartMode_Privileged** (uint8 instance)
 - Set node to start mode.
- Flexcan_Ip_StatusType **FlexCAN_Ip_GetTransferStatus**(uint8 instance, uint8 mb_idx)
 - Return the mb's state: busy (in progress) or idle(success).
- boolean **FlexCAN_Ip_GetBuffStatusFlag**(uint8 instance, uint8 msgBuffIdx)
 - Read the IFLAG and return true if there is an interrupt happen.
- Flexcan_Ip_StatusType **FlexCAN_Ip_SetTDCOffset_Privileged** (uint8 instance, boolean enable, uint8 offset)
 - Enter freeze mode first and set the ETDCOFF in ETDC register or TDCOFF value in FDCTRL register.
- Flexcan_Ip_StatusType **FlexCAN_Ip_SetTxArbitrationStartDelay** (uint8 instance, uint8 value)
 - Enter freeze mode first and set the TASD value in CTRL2.
- Flexcan_Ip_StatusType **FlexCAN_Ip_SetStopMode** (uint8 instance)
- Flexcan_Ip_StatusType **FlexCAN_Ip_SetBitrate_Privileged**(uint8 instance, const Flexcan_Ip_TimeSegmentType * bitrate, boolean enhExt)

KEY FUNCTION – SENDING

- Flexcan_Ip_StatusType **FlexCAN_Ip_Send**(uint8 instance,uint8 mb_idx,const Flexcan_Ip_DataInfoType * tx_info,uint32 msg_id,const uint8 * mb_data);
 - FlexCAN_Ip.c
 - Sending a CAN frame and enable mb interrupt in corresponding IMASK register after successful transmission if sends by interrupt method.
- void **FlexCAN_Ip_MainFunctionWrite**(uint8 instance, uint8 mb_idx);
 - Clear the interrupt flag and transform the mb state to be idle.
- Flexcan_Ip_StatusType **FlexCAN_Ip_SendBlocking**(uint8 instance,uint8 mb_idx,const Flexcan_Ip_DataInfoType * tx_info,uint32 msg_id,const uint8 * mb_data,uint32 timeout_ms);
 - Sending a CAN frame, blocks until either the frame was sent, or timeout expired.
- static Flexcan_Ip_StatusType **FlexCAN_StartSendData**(uint8 Flexcan_Ip_u8Instance,uint8 mb_idx,const Flexcan_Ip_DataInfoType * tx_info,uint32 msg_id,const uint8 * mb_data);
 - Start a transmit by beginning the process of sending data. State's structure to busy and invoke FlexCAN_SetTxMsgBuff to write the cs and data in physical mb address.
- Flexcan_Ip_StatusType **FlexCAN_SetTxMsgBuff**(FLEXCAN_Type * base, uint32 msgBuffIdx, const Flexcan_Ip_MsbuffCodeStatusType * cs, uint32 msgId, const uint8 * msgData, const boolean isRemote)
 - A CAN frame will be sent immediately at the end of this function where cs code was wrote to the address.
- Flexcan_Ip_StatusType **FlexCAN_Ip_AbortTransfer**(uint8 u8Instance, uint8 mb_idx)
 - Write the cs code to abort mb transfer, clear the interrupt flag and turn the mb state to be idle.

KEY FUNCTION – RECEIVING

- Flexcan_Ip_StatusType **FlexCAN_Ip_ConfigRxMb**(uint8 instance, uint8 mb_idx, const Flexcan_Ip_DataInfoType * rx_info, uint32 msg_id)
 - FlexCAN_Ip.c
 - Write the cs code and id in mailbox physical address to initialize receive mb.
- Flexcan_Ip_StatusType **FlexCAN_Ip_Receive**(uint8 instance, uint8 mb_idx, Flexcan_Ip_MsgBuffType * data, boolean isPolling)
 - Call FlexCAN_StartRxMessageBufferData to start receiving, enable the interrupt in IMASK register if needed.
- Static Flexcan_Ip_StatusType **FlexCAN_StartRxMessageBufferData**(uint8 instance, uint8 mb_idx, Flexcan_Ip_MsgBuffType * data, Boolean isPolling)
 - Transform the state structure to be busy and start to receive.
- Flexcan_Ip_StatusType **FlexCAN_Ip_ReceiveBlocking**(uint8 instance, uint8 mb_idx, Flexcan_Ip_MsgBuffType * data, boolean isPolling, uint32 u32TimeoutMs)
- void **FlexCAN_Ip_MainFunctionRead**(uint8 instance, uint8 mb_idx)
 - Read the messages received as polling if the interrupt is disabled. Invoke the corresponding IRQHandler to process receiving.
- Flexcan_Ip_StatusType **FlexCAN_Ip_SetRxMaskType_Privileged**(uint8 instance, Flexcan_Ip_RxMaskType type)
 - Choose the mask type.(MCR[IRMQ], '0' means global mask , '1' means individual mask)
- Flexcan_Ip_StatusType **FlexCAN_Ip_SetRxMbGlobalMask_Privileged**(uint8 instance, uint32 mask)
 - Set Rx message buffer global mask.
- Flexcan_Ip_StatusType **FlexCAN_Ip_SetRxIndividualMask_Privileged**(uint8 instance, uint8 mb_idx, uint32 mask)
 - Set individual mask in RXIMR register.

KEY FUNCTION – FIFO

- void **FlexCAN_Ip_ConfigEnhancedRxFifo_Privileged**(uint8 instance, const Flexcan_Ip_EnhancedIdTableType * id_filter_table)
 - Write the user configured enhanced FIFO filter table to ERFEL register.
- Flexcan_Ip_StatusType **FlexCAN_Ip_ConfigRxFifo_Privileged**(uint8 instance, Flexcan_Ip_RxFIFOIdElementFormatType id_format, const Flexcan_Ip_IdTableType * id_filter_table)
 - Configure the filter table elements in legacy FIFO.
- Flexcan_Ip_StatusType **FlexCAN_Ip_RxFifo** (uint8 instance, Flexcan_Ip_MsgBuffType * data)
 - Invoke FlexCAN_StartRxMessageFIFOData or FlexCAN_StartRxMessageEnhancedFIFOData to start FIFO receiving.
- Flexcan_Ip_StatusType **FlexCAN_Ip_RxFifoBlocking**(uint8 instance, Flexcan_Ip_MsgBuffType *data, uint32 timeout)
 - Start to receive FIFO by blocking method.
- static Flexcan_Ip_StatusType **FlexCAN_StartRxMessageEnhancedFIFOData**(uint8 instance, Flexcan_Ip_MsgBuffType * data)
 - Transform enhanced FIFO state structure to busy and enable, and make the state output message address equal to user receiving buffer.
- Flexcan_Ip_StatusType **FlexCAN_Ip_SetRxFIFOGlobalMask_Privileged**(uint8 instance, uint32 mask)
 - Set legacy FIFO global mask (RXFGMASK)
- Flexcan_Ip_StatusType **FlexCAN_Ip_SetRxMaskType_Privileged**(uint8 instance, Flexcan_Ip_RxMaskType type)
 - Choose the mask type.(MCR[IRMQ], '0' means global mask , '1' means individual mask)

KEY FUNCTION – INTERRUPT/BUS OFF

- void **FlexCAN_IRQHandler**(uint8 instance, uint8 startMbIdx, uint8 endMbIdx)
 - Process mb transmit/receive interrupt. Put the state's structure to be idle and clear the flag in IFLAG register.
- static void **FlexCAN_IRQHandlerRxMB**(uint8 instance, uint32 mb_idx)
 - Lock the mailbox and get the data in physical address and call the callback function, etc.
- static inline void **FlexCAN_IRQHandlerRxFIFO**(uint8 instance, uint32 mb_idx)
 - If there is a frame available in FIFO, read the data in FIFO and clear the flag. Call the callback function and transform the state structure to idle. Disable the FIFO interrupt in IMASK register. Other circumstance, will only invoke the callback function.
- void **FlexCAN_IRQHandlerEnhancedRxFIFO**(uint8 instance, uint32 intType)
 - Read enhanced FIFO output and clear the flag(ERFSR), invoke callback function and disable the enhanced FIFO interrupt.
- void **FlexCAN_BusOff_IRQHandler**(uint8 instance)
 - Only get the bus off flag (ESR1[BOFFINT]) and call the callback function, then it will clear the interrupt flag.
- void **FlexCAN_Busoff_Error_IRQHandler**(uint8 instance)
 - Clear the ESR1 interrupt flag and call the error callback.
- Flexcan_Ip_StatusType **FlexCAN_Ip_MainFunctionBusOff_Privileged**(uint8 instance)
 - Read the ESR bus off flag and invoke the callback, then clear the flag and set node to stop mode.

CAN TX RX (USE CASE)



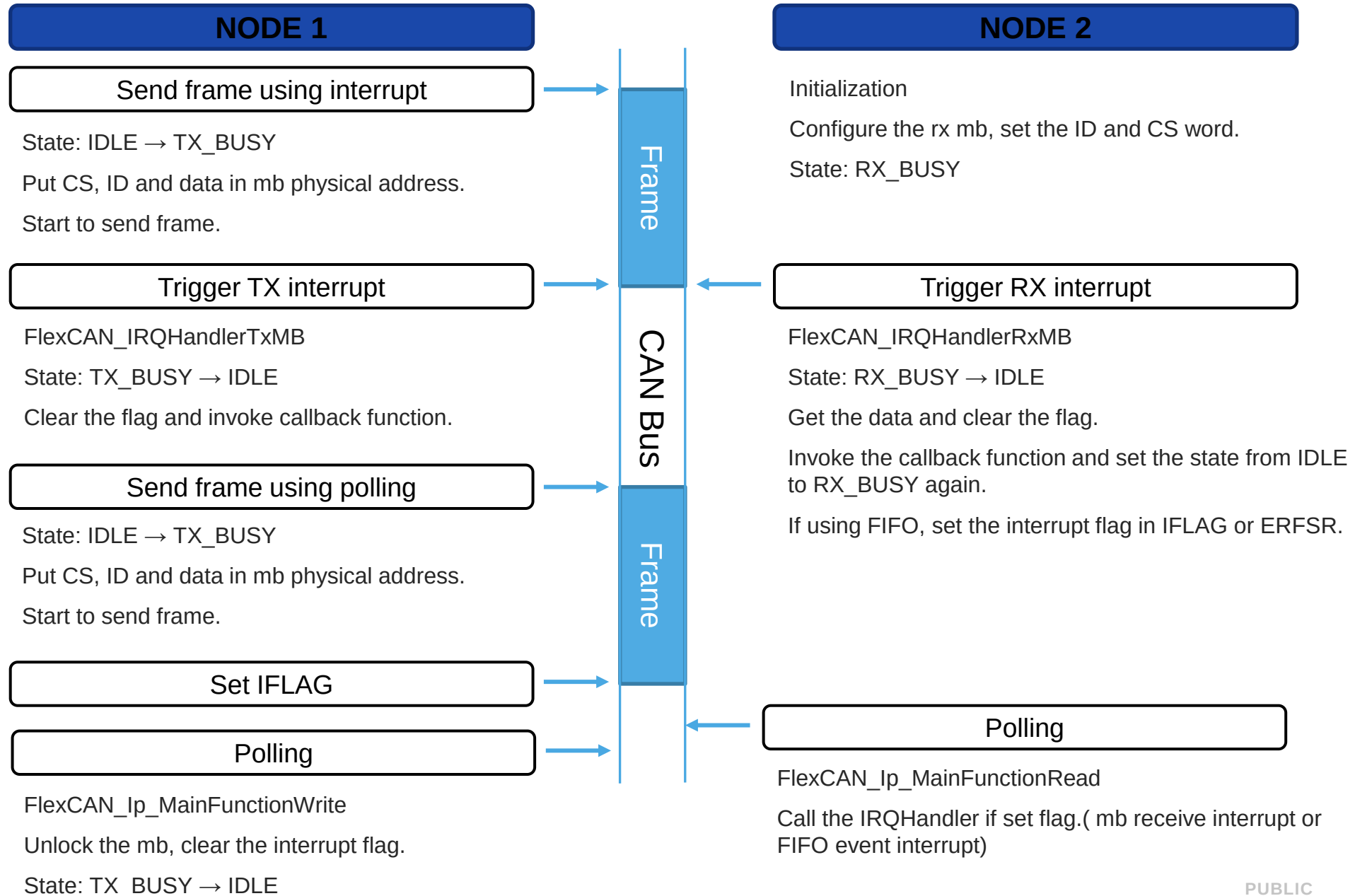
SECURE CONNECTIONS
FOR A SMARTER WORLD

PUBLIC

NXP, THE NXP LOGO AND NXP SECURE CONNECTIONS FOR A SMARTER WORLD ARE TRADEMARKS OF NXP B.V.
ALL OTHER PRODUCT OR SERVICE NAMES ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS. © 2021 NXP B.V.



INTERRUPT VS POLLING



USE CASE

Example 1: Legacy FIFO feature

1. Declare the filter table elements in 8 filter table elements.

`FLEXCAN_RX_FIFO_ID_FORMAT_A`

```
const Flexcan_Ip_IdTableType CAN2_LegacyFIFO_IdFilterTable[8] =
{
    /* Legacy FIFO filter table element 0-7 */
    {.id = 0x55, .isExtendedFrame = false, .isRemoteFrame = false,},
    .
    .
    .
    {.id = 0x145, .isExtendedFrame = true, .isRemoteFrame = false,},
};
```

2. Select the mask type to individual mask.

```
FlexCAN_Ip_SetRxMaskType_Privileged(INST_FLEXCAN_2, FLEXCAN_RX_MASK_INDIVIDUAL);
```

3. Select the format and configure the FIFO filter.

```
/* configure the FlexCAN RxFIFO filter table */
FlexCAN_Ip_ConfigRxFIFO_Privileged(INST_FLEXCAN_2,
FLEXCAN_RX_FIFO_ID_FORMAT_A, &CAN2_LegacyFIFO_IdFilterTable[0]);
```

4. Set the individual mask or global mask.

```
/* configure the FlexCAN RxFIFO filter table ID mask */
for(FIFOfilter_idx=0; FIFOfilter_idx < FIFOfilter_maxnum; FIFOfilter_idx++)
{
    /* mask all the bits */
    FlexCAN_Ip_SetRxIndividualMask_Privileged(INST_FLEXCAN_2, FIFOfilter_idx, MB_INDMASK(FIFOfilter_idx));
}
```

`FLEXCAN_RX_FIFO_ID_FORMAT_B`

```
const Flexcan_Ip_IdTableType CAN2_LegacyFIFO_IdFilterTable[16] =
{
    /* Legacy FIFO filter table element 0-15 */
    {.id = 0x55, .isExtendedFrame = false, .isRemoteFrame = false,},
    .
    .
    .
    {.id = 0x145, .isExtendedFrame = true, .isRemoteFrame = false,},
};
```

5. Start to receive.

```
/* open the RX enhanced or legacy FIFO */
FlexCAN_Ip_RxFIFO(INST_FLEXCAN_2, &CAN2_FIFO_Received_MB[0]);
```

USE CASE

Example 1 : Legacy FIFO feature

`FLEXCAN_RX_FIFO_ID_FORMAT_A`

For standard ID: don't care the 4 least significant bits. For extended ID: don't care bits 4-8.

num	ID	Individual mask	Filter elements ram binary
1	0x55 standard frame	(0x7F0<<19) IDE RTR	00 000 0101 0101 00 0000 0000 0000 0000 0
2	0x65 standard frame	(0x7F0<<19) IDE RTR	00 000 0110 0101 00 0000 0000 0000 0000 0
3	0x75 standard frame	(0x7F0<<19) IDE RTR	00 000 0111 0101 00 0000 0000 0000 0000 0
4	0x85 standard frame	(0x7F0<<19) IDE RTR	00 000 1000 0101 00 0000 0000 0000 0000 0
5	0x115 extended frame	(0x1FFFFFF0F<<1) IDE RTR	01 000 0000 0000 00 0000 0001 0001 0101 0
6	0x215 extended frame	(0x1FFFFFF0F<<1) IDE RTR	01 000 0000 0000 00 0000 0010 0001 0101 0
7	0x315 extended frame	(0x1FFFFFF0F<<1) IDE RTR	01 000 0000 0000 00 0000 0011 0001 0101 0
8	0x415 extended frame	(0x1FFFFFF0F<<1) IDE RTR	01 000 0000 0000 00 0000 0100 0001 0101 0

- A 2 bits for RTR and IDE
- A 11 bits for standard ID comparison
- A+A 29 bits for extended ID comparison
- A 1 reserved bit

Acceptable standard IDs:	Acceptable extended IDs:
0x5X (hit element 0)	0x1X5 (hit element 4)
0x6X (hit element 1)	0x2X5 (hit element 5)
0x7X (hit element 2)	0x3X5 (hit element 6)
0x8X (hit element 3)	0x4X5 (hit element 7)

0000 0000	RXIMR[0]	0xFF800000	11111111100000000000000000000000
0000 0000	RXIMR[1]	0xFF800000	11111111100000000000000000000000
0000 0000	RXIMR[2]	0xFF800000	11111111100000000000000000000000
0000 0000	RXIMR[3]	0xFF800000	11111111100000000000000000000000
0000 0000	RXIMR[4]	0xFFFFFE1E	1111111111111111111111111000011110
0000 0000	RXIMR[5]	0xFFFFFE1E	1111111111111111111111111000011110
0000 0000	RXIMR[6]	0xFFFFFE1E	1111111111111111111111111000011110
0000 0000	RXIMR[7]	0xFFFFFE1E	1111111111111111111111111000011110

Attention: Compare to K1 SDK, RTD 0.9.0 FlexCAN_Ip_SetRxIndividualMask_Privileged will not put the individual mask in expected bits, rather it just put the mask in RXIMR register.

USE CASE

Example 1 : Legacy FIFO feature

FLEXCAN_RX_FIFO_ID_FORMAT_A

19-29bits are matched for standard ID, 1-29 bits are matched for extended ID, 0bit is reserved, 30-31bits matched for RTR and IDE.

num	ID	Individual mask	Filter elements ram binary
1	0x55 standard frame	(0x7F0<<19) IDE RTR	00 000 0101 0101 00 0000 0000 0000 0000 0
2	0x65 standard frame	(0x7F0<<19) IDE RTR	00 000 0110 0101 00 0000 0000 0000 0000 0
3	0x75 standard frame	(0x7F0<<19) IDE RTR	00 000 0111 0101 00 0000 0000 0000 0000 0
4	0x85 standard frame	(0x7F0<<19) IDE RTR	00 000 1000 0101 00 0000 0000 0000 0000 0
5	0x115 extended frame	(0x1FFFFFF0F<<1) IDE RTR	01 000 0000 0000 00 0000 0001 0001 0101 0
6	0x215 extended frame	(0x1FFFFFF0F<<1) IDE RTR	01 000 0000 0000 00 0000 0010 0001 0101 0
7	0x315 extended frame	(0x1FFFFFF0F<<1) IDE RTR	01 000 0000 0000 00 0000 0011 0001 0101 0
8	0x415 extended frame	(0x1FFFFFF0F<<1) IDE RTR	01 000 0000 0000 00 0000 0100 0001 0101 0

- A 2 bits for RTR and IDE
- A 11 bits for standard ID comparison
- A+A 29 bits for extended ID comparison
- A 1 reserved bit

For standard ID: don't care the 4 least significant bits. For extended ID: don't care bits 4-8.

055h	8	55 55
06Ah	2	66 66
081h	8	88 88
00000135h	6	13 13
00000455h	2	45 45
00000179h	2	17 09
093h	5	93 93
00000509h	5	50 95

Acceptable

Not accept

CAN2_FIFO_Received_MB[0]	Flexcan_Ip_MsgBuffType	{...}	CAN2_FIFO_Received_MB[0]	Flexcan_Ip_MsgBuffType	{...}
cs	uint32	0x3e21b09 (Hex)	cs	uint32	0x8f150 (Hex)
msgId	uint32	0x455 (Hex)	msgId	uint32	0x55 (Hex)
> data	uint8 [64]	0x20401df0 (Hex)	> data	uint8 [64]	0x20401df0 (Hex)
dataLen	uint8	0x2 (Hex)	dataLen	uint8	0x8 (Hex)
id_hit	uint8	0x7 (Hex)	id_hit	uint8	0x0 (Hex)
time_stamp	uint32	0x1b09 (Hex)	time_stamp	uint32	0xf150 (Hex)
CAN2_FIFO_Received_MB[0]	Flexcan_Ip_MsgBuffType	{...}	CAN2_FIFO_Received_MB[0]	Flexcan_Ip_MsgBuffType	{...}
cs	uint32	0x1886786 (Hex)	cs	uint32	0x266dbcc (Hex)
msgId	uint32	0x81 (Hex)	msgId	uint32	0x135 (Hex)
> data	uint8 [64]	0x20401df0 (Hex)	> data	uint8 [64]	0x20401df0 (Hex)
dataLen	uint8	0x8 (Hex)	dataLen	uint8	0x6 (Hex)
id_hit	uint8	0x3 (Hex)	id_hit	uint8	0x4 (Hex)
time_stamp	uint32	0x6786 (Hex)	time_stamp	uint32	0xdbcc (Hex)

USE CASE

Example 1 : Legacy FIFO feature

FLEXCAN_RX_FIFO_ID_FORMAT_A

19-29bits are matched for standard ID, 1-29 bits are matched for extended ID, 0bit is reserved, 30-31bits matched for RTR and IDE.

num	ID	Individual mask	Filter elements ram binary
1	0x55 standard frame	(0x7F0<<19) IDE RTR	00 000 0101 0101 00 0000 0000 0000 0000 0
2	0x65 standard frame	(0x7F0<<19) IDE RTR	00 000 0110 0101 00 0000 0000 0000 0000 0
3	0x75 standard frame	(0x7F0<<19) IDE RTR	00 000 0111 0101 00 0000 0000 0000 0000 0
4	0x85 standard frame	(0x7F0<<19) IDE RTR	00 000 1000 0101 00 0000 0000 0000 0000 0
5	0x115 extended frame	(0x1FFFFFF0F<<1) IDE RTR	01 000 0000 0000 00 0000 0001 0001 0101 0
6	0x215 extended frame	(0x1FFFFFF0F<<1) IDE RTR	01 000 0000 0000 00 0000 0010 0001 0101 0
7	0x315 extended frame	(0x1FFFFFF0F<<1) IDE RTR	01 000 0000 0000 00 0000 0011 0001 0101 0
8	0x415 extended frame	(0x1FFFFFF0F<<1) IDE RTR	01 000 0000 0000 00 0000 0100 0001 0101 0

- A 2 bits for RTR and IDE
- A 11 bits for standard ID comparison
- A+A 29 bits for extended ID comparison
- A 1 reserved bit

For standard ID: don't care the 4 least significant bits. For extended ID: don't care bits 4-8.

055h	8	55 55
06Ah	2	66 66
081h	8	88 88
00000135h	6	13 13
00000455h	2	45 45
00000179h	2	17 09
093h	5	93 93
00000509h	5	50 95

Acceptable

Not accept

CAN2_FIFO_Received_MB[0]	Flexcan_Ip_MsgBuffType	{...}	CAN2_FIFO_Received_MB[0]	Flexcan_Ip_MsgBuffType	{...}
cs	uint32	0x3e21b09 (Hex)	cs	uint32	0x8f150 (Hex)
msgId	uint32	0x455 (Hex)	msgId	uint32	0x55 (Hex)
> data	uint8 [64]	0x20401df0 (Hex)	> data	uint8 [64]	0x20401df0 (Hex)
dataLen	uint8	0x2 (Hex)	dataLen	uint8	0x8 (Hex)
id_hit	uint8	0x7 (Hex)	id_hit	uint8	0x0 (Hex)
time_stamp	uint32	0x1b09 (Hex)	time_stamp	uint32	0xf150 (Hex)
CAN2_FIFO_Received_MB[0]	Flexcan_Ip_MsgBuffType	{...}	CAN2_FIFO_Received_MB[0]	Flexcan_Ip_MsgBuffType	{...}
cs	uint32	0x1886786 (Hex)	cs	uint32	0x266dbcc (Hex)
msgId	uint32	0x81 (Hex)	msgId	uint32	0x135 (Hex)
> data	uint8 [64]	0x20401df0 (Hex)	> data	uint8 [64]	0x20401df0 (Hex)
dataLen	uint8	0x8 (Hex)	dataLen	uint8	0x6 (Hex)
id_hit	uint8	0x3 (Hex)	id_hit	uint8	0x4 (Hex)
time_stamp	uint32	0x6786 (Hex)	time_stamp	uint32	0xdbcc (Hex)

USE CASE

Example 1 : Legacy FIFO feature

FLEXCAN_RX_FIFO_ID_FORMAT_B

19-29, 3-13bits are matched for standard ID. 16-29, 0-13 bits are matched for extended ID 14 most significant bits(16-29). 30-31, 14-15bits matched for RTR and IDE.

num	ID	Individual mask	Filter elements ram binary
1	0x55 standard frame	0x7f877f87	00 000 0101 0101 000 00 000 0110 0101 000
2	0x65 standard frame		
3	0x75 standard frame		
4	0x85 standard frame		
5	0x95 standard frame		
6	0xA5 standard frame	0x7f877f87	00 000 1001 0101 000 00 000 1010 0101 000
7	0xB5 standard frame		
8	0xC5 standard frame		
9	0x1150000 extended frame		
10	0x2150000 extended frame		
11	0x3150000 extended frame	0x7fe17fe1	01 0 0001 0001 0101 0 01 0 0010 0001 0101 0 29 25-28 21-24 17-20 16
12	0x4150000 extended frame		
13	0x5150000 extended frame		
14	0x6150000 extended frame		
15	0x7150000 extended frame		
16	0x8150000 extended frame	0x7fe17fe1	01 0 0111 0001 0101 0 01 0 1000 0001 0101 0

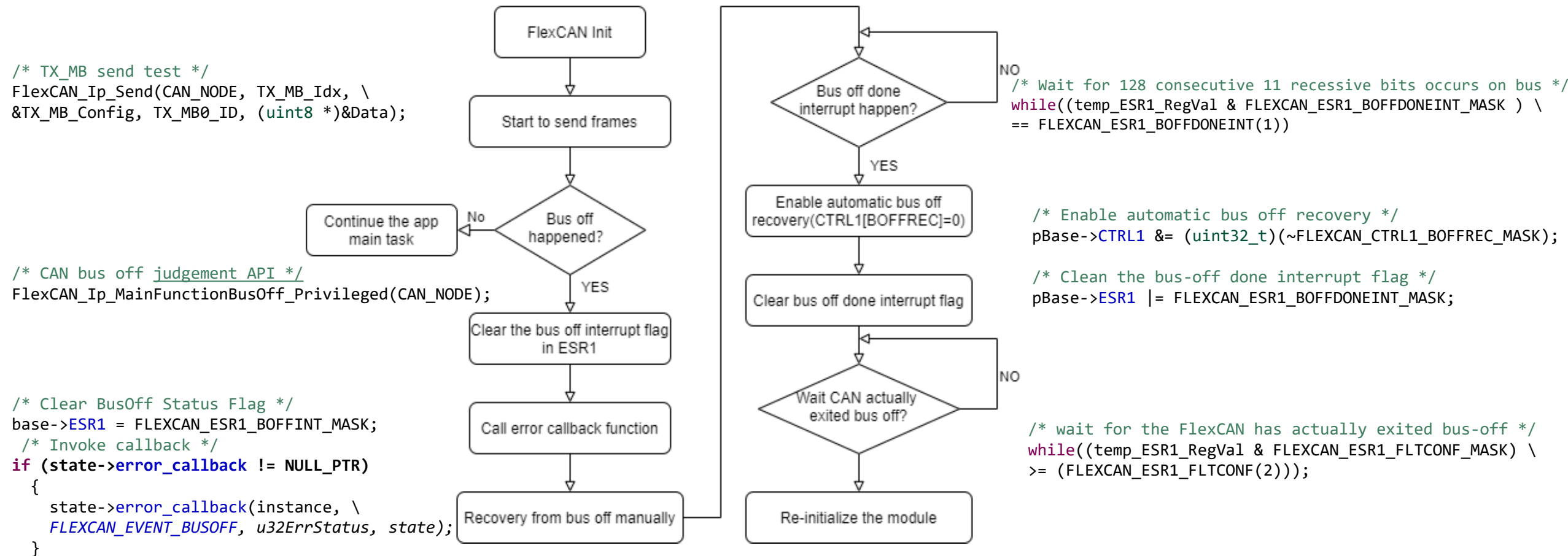
- A 2 bits for RTR and IDE
- A 11 bits for standard ID comparison
- A+A 14 bits for extended ID comparison
- A 1 reserved bit

For standard ID: don't care the 4 least significant bits. For extended ID: don't care bits 17-20(start from 1).

RXIMR[0]	0x7F877F87	01111111100001110111111110000111
RXIMR[1]	0x7F877F87	01111111100001110111111110000111
RXIMR[2]	0x7F877F87	01111111100001110111111110000111
RXIMR[3]	0x7F877F87	01111111100001110111111110000111
RXIMR[4]	0x7FE17FE1	0111111111000010111111111100001
RXIMR[5]	0x7FE17FE1	0111111111000010111111111100001
RXIMR[6]	0x7FE17FE1	0111111111000010111111111100001
RXIMR[7]	0x7FE17FE1	0111111111000010111111111100001

USE CASE

- Example 2 : poll bus off status and recovery manually.



USE CASE

- Example 3 : queue when CPU not serve the mb.

Configure MB0-2 to receive the same id frame :

055h	8	88 88 88 88 88 88 88 88
------	---	-------------------------

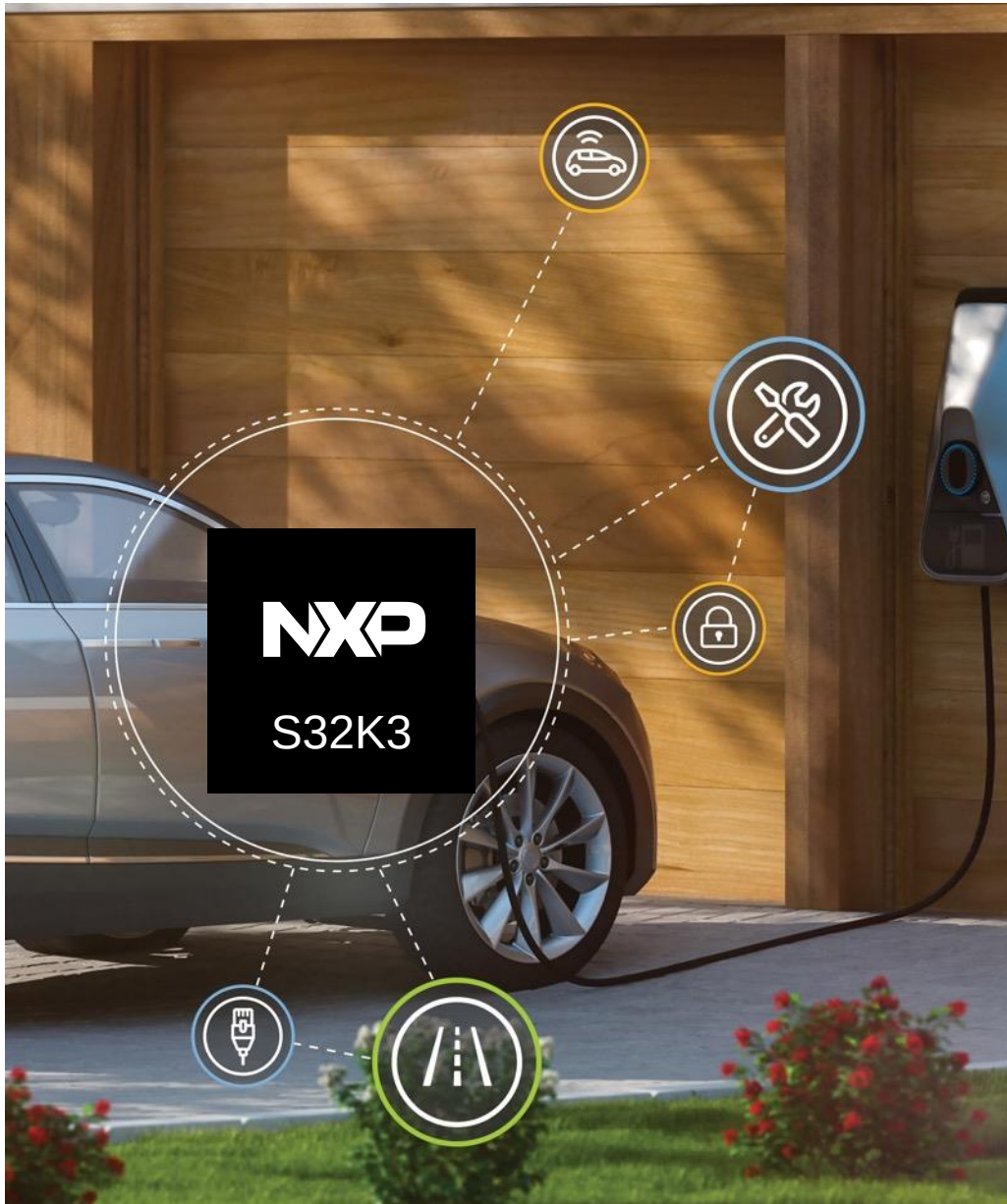
```
/* Configure the mb0-3 to receive frame */
FlexCAN_Ip_ConfigRxMb(CAN_NODE, RX_MB0, &RX_MB0_Config, MB_ID);
FlexCAN_Ip_ConfigRxMb(CAN_NODE, RX_MB1, &RX_MB1_Config, MB_ID);
FlexCAN_Ip_ConfigRxMb(CAN_NODE, RX_MB2, &RX_MB2_Config, MB_ID);
```

```
FlexCAN_Ip_Receive(CAN_NODE, RX_MB0, &Received_MB[0], true);
FlexCAN_Ip_Receive(CAN_NODE, RX_MB1, &Received_MB[1], true);
FlexCAN_Ip_Receive(CAN_NODE, RX_MB2, &Received_MB[2], true);
```

1. The physical memory of MB0-2 before frames coming.
code field: 0x4, empty state
2. Frame1 comes and accepted by mb0. The code field of mb0 turns to 0x2(full state).
3. Frame2 comes and accepted by mb1. The code field of mb1 turns to 0x2(full state).
4. Frame3 comes and accepted by mb2. The code field of mb2 turns to 0x2.
5. Frame4 comes and accepted by mb2 again. Turn the code field to 0x6(Overrun state).

1	Address	0 - 3	4 - 7	8 - B	C - F	
	40308080	00000004	00005401	00000000	00000000	
	40308090	00000004	00005401	00000000	00000000	
	403080A0	00000004	00005401	00000000	00000000	
2	Address	0 - 3	4 - 7	8 - B	C - F	
	40308080	5D2A0802	00005401	88888888	88888888	Frame 1
	40308090	00000004	00005401	00000000	00000000	
	403080A0	00000004	00005401	00000000	00000000	
3	Address	0 - 3	4 - 7	8 - B	C - F	
	40308080	A0FB0802	00005401	88888888	88888888	
	40308090	91DE0802	00005401	88888888	88888888	Frame 2
	403080A0	00000004	00005401	00000000	00000000	
4	Address	0 - 3	4 - 7	8 - B	C - F	
	40308080	98B10802	00005401	88888888	88888888	
	40308090	E18D0802	00005401	88888888	88888888	
	403080A0	11F90802	00005401	88888888	88888888	Frame 3
5	Address	0 - 3	4 - 7	8 - B	C - F	
	40308080	D1860802	00005401	88888888	88888888	
	40308090	4DFC0802	00005401	88888888	88888888	
	403080A0	43120806	00005401	88888888	88888888	Frame 4

CODE field



S32K3 MICROCONTROLLERS FOR AUTOMOTIVE AND INDUSTRIAL

Tackling cost and complexity of automotive software development



[S32K MCU family page](#)

[S32K3 MCU page](#)



Online engineering support:

- [S32K MCUs community](#)



SECURE CONNECTIONS
FOR A SMARTER WORLD



[SHOWROOM.NXP.COM](https://www.showroom.nxp.com)